

Graph & Searching Algorithms

Comprehensive Guide Covering BFS, DFS, Advanced Traversals, and Pathfinding Problems

1. Breadth-First Search (BFS) Algorithms

Problem 1.1: Binary Tree Level Order Traversal (BFS)

Given the root of a binary tree, return the level order traversal of its nodes' values (i.e., from left to right, level by level).

Time Complexity: $O(V)$

Space Complexity: $O(V)$

```
from collections import deque

def levelOrder(root: TreeNode) -> list[list[int]]:
    res = []
    if not root:
        return res

    queue = deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        res.append(level)
    return res
```

Problem 1.2: Shortest Path in a Binary Matrix (Grid BFS)

Given an $n \times n$ binary grid, return the length of the shortest clear path from the top-left cell to the bottom-right cell. If no path exists, return -1.

Time Complexity: $O(N \times M)$

Space Complexity: $O(N \times M)$

```
def shortestPathBinaryMatrix(grid: list[list[int]]) -> int:
    n = len(grid)
    if grid[0][0] == 1 or grid[n-1][n-1] == 1:
        return -1

    queue = deque([(0, 0, 1)]) # r, c, length
    visit = {(0, 0)}
    directions = [(0,1),(0,-1),(1,0),(-1,0),(1,1),(-1,-1),(1,-1),(-1,1)]

    while queue:
        r, c, d = queue.popleft()
        if r == n - 1 and c == n - 1:
            return d
        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            if 0 <= nr < n and 0 <= nc < n and grid[nr][nc] == 0 and (nr, nc) not in visit:
                queue.append((nr, nc, d + 1))
                visit.add((nr, nc))

    return -1
```

2. Depth-First Search (DFS) Algorithms

Problem 2.1: Number of Islands (Grid Connected Components)

Given an $m \times n$ 2D binary grid which represents a map of '1's (land) and '0's (water), return the number of islands discovered using recursive DFS.

Time Complexity: $O(V + E)$ or $O(M \times N)$

Space Complexity: $O(M \times N)$ (Call Stack size)

```
def numIslands(grid: list[list[str]]) -> int:
    if not grid: return 0
    rows, cols = len(grid), len(grid[0])
    count = 0

    def dfs(r, c):
        if r < 0 or c < 0 or r >= rows or c >= cols or grid[r][c] == '0':
            return
        grid[r][c] = '0' # Sink land to avoid revisiting
        dfs(r + 1, c)
        dfs(r - 1, c)
        dfs(r, c + 1)
        dfs(r, c - 1)

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                count += 1
                dfs(r, c)

    return count
```

3. Advanced Graph Algorithms & Pathfinding

Problem 3.1: Course Schedule (Topological Sort / Cycle Detection)

There are a total of `numCourses` courses you have to take, labeled from 0 to `numCourses - 1`. Some courses have prerequisites. Return `true` if you can finish all courses.

Time Complexity: $O(V + E)$

Space Complexity: $O(V + E)$

```
def canFinish(numCourses: int, prerequisites: list[list[int]]) -> bool:
    adj = {i: [] for i in range(numCourses)}
    for crs, pre in prerequisites:
        adj[crs].append(pre)

    visiting = set() # Active path recursion stack tracker

    def dfs(crs):
        if crs in visiting: return False # Cycle found
        if adj[crs] == []: return True

        visiting.add(crs)
        for pre in adj[crs]:
            if not dfs(pre): return False
        visiting.remove(crs)
        adj[crs] = [] # Optimization memoization
        return True

    for c in range(numCourses):
        if not dfs(c): return False
    return True
```

Problem 3.2: Network Delay Time (Dijkstra's Algorithm)

You are given a network of `n` nodes. You are also given `times`, a list of travel times directed edges. Return the minimum time it takes for all nodes to receive a signal sent from node `k`.

Time Complexity: $O(E \times \log(V))$

Space Complexity: $O(V + E)$

```

import heapq

def networkDelayTime(times: list[list[int]], n: int, k: int) -> int:
    edges = {i: [] for i in range(1, n + 1)}
    for u, v, w in times:
        edges[u].append((v, w))

    min_heap = [(0, k)] # cost, node
    visit = set()
    t = 0

    while min_heap:
        w1, n1 = heapq.heappop(min_heap)
        if n1 in visit: continue
        visit.add(n1)
        t = max(t, w1)

        for n2, w2 in edges[n1]:
            if n2 not in visit:
                heapq.heappush(min_heap, (w1 + w2, n2))

    return t if len(visit) == n else -1

```